# Section 16: Graphs

This section is part of the 5-hour LeetCode DSA Foundations course by Mukul Raina

- YouTube Course: https://youtu.be/YJMBv4Q2Tg8?si=Q1vDzusUJf753GG3
- YouTube Channel: https://www.youtube.com/@mukulraina
- Instructor LinkedIn: https://www.linkedin.com/in/mukulraina/


SLIDE 150: Section Intro, Graphs

SLIDE 151: Graph Representations: Adjacency List and Adjacency Matrix

SLIDE 152: Directed vs Undirected, Weighted vs Unweighted

SLIDE 153: BFS on Graphs: Level-by-Level Exploration

SLIDE 154: DFS on Graphs: Depth-First Exploration

SLIDE 155: Worked Example: Number of Islands

SLIDE 156: Connected Components

SLIDE 157: Cycle Detection in Graphs

SLIDE 158: Wrong vs Right: Graph Mistakes

SLIDE 159: How to Present Graph Solutions in an Interview

SLIDE 160: Interview Practice: Graphs

SLIDE 161: Section Cheat Sheet


## SLIDE 150: Section Intro, Graphs

**Section 16: Graphs Foundations**

Graphs generalize trees. Where a tree has a single root, no cycles, and exactly one path between any two nodes, a graph allows cycles, multiple paths, disconnected components, and directed or undirected edges. Many real-world problems that do not look like graph problems turn out to be graph problems: navigating a grid, detecting dependencies, finding connected groups, checking reachability.

Graph problems appear in roughly 15-20% of DSA interviews. Nearly every graph problem reduces to one of two fundamental traversals: BFS (breadth-first search) or DFS (depth-first search). If you know when to apply each and how to implement them cleanly, you can solve the majority of graph interview questions.

**What We'll Cover**

1. Graph representations: adjacency list and adjacency matrix
2. Directed vs undirected, weighted vs unweighted
3. BFS: level-by-level exploration with shortest path guarantee
4. DFS: depth-first exploration, recursive and iterative
5. Grids as implicit graphs
6. Connected components and cycle detection
7. Worked example: Number of Islands

**Where This Section Connects**

| Section | Connection |
|---|---|
| Stacks and Queues (Section 10) | BFS uses a queue, iterative DFS uses a stack. The data structure drives the traversal order. |
| Linked Lists (Section 11) | Floyd's cycle detection on linked lists is a specialized form of cycle detection. Graph cycle detection generalizes it. |
| Recursion (Section 12) | Recursive DFS is a direct application of the recursive mental model from Section 12. |
| Trees (Section 15) | Trees are a special case of graphs (connected, acyclic). Tree BFS and DFS are graph BFS and DFS without a visited set. |

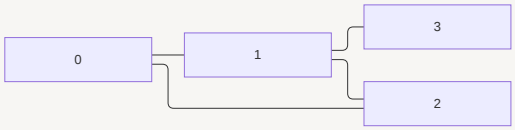
Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

## SLIDE 151: Graph Representations: Adjacency List and Adjacency Matrix

### How to Store a Graph in Code

A graph has V vertices (nodes) and E edges (connections). Before solving any graph problem, you need to decide how to represent these edges in memory. Two standard representations exist, and the choice affects both space usage and operation costs.




### Adjacency List

Each node maps to a list of its neighbors. Space-efficient for sparse graphs (few edges relative to nodes), which is the case for most interview problems.

```java
Map<Integer, List<Integer>> buildAdjList(int n, int[][] edges) {
    Map<Integer, List<Integer>> graph = new HashMap<>();

    for (int i = 0; i < n; i++) {
        graph.put(i, new ArrayList<>());
    }

    for (int[] edge : edges) {
        graph.get(edge[0]).add(edge[1]);
        graph.get(edge[1]).add(edge[0]);
    }

    return graph;
}
```

For the graph above with edges [[0,1],[0,2],[1,2],[1,3]], the adjacency list is:

```
0: [1, 2]
1: [0, 2, 3]
2: [0, 1]
3: [1]
```

**Adjacency Matrix**

A V x V boolean (or integer) matrix where matrix[i][j] = true means an edge exists from node i to node j.

```java
boolean[][] buildAdjMatrix(int n, int[][] edges) {
    boolean[][] matrix = new boolean[n][n];

    for (int[] edge : edges) {
        matrix[edge[0]][edge[1]] = true;
        matrix[edge[1]][edge[0]] = true;
    }

    return matrix;
}
```

**Comparison Table**

| Property | Adjacency List | Adjacency Matrix |
|---|---|---|
| Space | O(V + E) | O(V^2) |
| Check if edge exists | O(degree of node) | O(1) |
| Get all neighbors | O(degree of node) | O(V) |
| Add edge | O(1) | O(1) |
| Best for | Sparse graphs (E much less than V^2) | Dense graphs (E close to V^2) |
| Interview default | Yes | Rarely |

**Default to adjacency list.** Most interview problems involve sparse graphs where E is proportional to V, not V^2. The adjacency list uses far less memory and iterating over neighbors is faster. Use the matrix only when the problem specifically benefits from O(1) edge lookups, which is uncommon in interviews.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

## SLIDE 152: Directed vs Undirected, Weighted vs Unweighted

### Undirected Graphs

An edge between A and B means you can travel in both directions. When building the adjacency list, add the neighbor in both directions.

```
graph.get(edge[0]).add(edge[1]);
graph.get(edge[1]).add(edge[0]);
```

### Directed Graphs

An edge from A to B means you can only travel from A to B, not from B to A. Add the neighbor in one direction only.

```
graph.get(edge[0]).add(edge[1]);
```

This single line difference between directed and undirected is easy to get wrong. Always clarify with the interviewer: "Are the edges directed or undirected?" before building the graph.

### Weighted Graphs

When edges have costs (distances, capacities, prices), store the weight alongside the neighbor. Use a pair or a small int array instead of a bare integer.

```java
Map<Integer, List<int[]>> buildWeightedGraph(int n, int[][] edges) {
    Map<Integer, List<int[]>> graph = new HashMap<>();

    for (int i = 0; i < n; i++) {
        graph.put(i, new ArrayList<>());
    }

    for (int[] edge : edges) {
        graph.get(edge[0]).add(new int[]{edge[1], edge[2]});
        graph.get(edge[1]).add(new int[]{edge[0], edge[2]});
    }

    return graph;
}
```

Each neighbor entry is {neighborNode, weight}. Edge input format: {from, to, weight}.

**What Interview Problems Typically Use**

| Graph Type | Frequency | Examples |
|---|---|---|
| Undirected, unweighted | Most common | Number of Islands, connected components, friend groups |
| Directed, unweighted | Common | Course prerequisites, dependency chains, topological sort |
| Undirected, weighted | Less common | Minimum spanning tree, network delay |
| Directed, weighted | Less common | Shortest path with costs (Dijkstra, preview for C5) |

Most interview graph problems at the foundations level are unweighted. When weights appear, the problem usually requires Dijkstra's algorithm or similar, which is covered in C5.

**Interview Tip:** Before writing any graph code, ask three questions: (1) "Are the edges directed or undirected?" (2) "Are the edges weighted?" (3) "Can the graph be disconnected?" These three questions determine your representation, your traversal, and whether you need an outer loop.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

## SLIDE 153: BFS on Graphs: Level-by-Level Exploration

**BFS Template**

BFS explores nodes in order of their distance from the source. It visits all nodes at distance 1 before any node at distance 2, all at distance 2 before any at distance 3, and so on. This makes BFS the correct choice whenever you need the shortest path in an unweighted graph.

The template uses a queue (FIFO) and a visited set. The queue ensures level-by-level ordering. The visited set prevents revisiting nodes, which is critical in graphs with cycles.

```
List<Integer> bfs(Map<Integer, List<Integer>> graph, int start) {
    List<Integer> order = new ArrayList<>();
    Set<Integer> visited = new HashSet<>();
    Queue<Integer> queue = new LinkedList<>();

    visited.add(start);
    queue.offer(start);

    while (!queue.isEmpty()) {
        int node = queue.poll();
        order.add(node);

        for (int neighbor : graph.get(node)) {
            if (!visited.contains(neighbor)) {
                visited.add(neighbor);
                queue.offer(neighbor);
            }
        }
    }

    return order;
}
```

**Critical difference from tree BFS:** trees have no cycles, so tree BFS (Section 15, Slide 141) does not need a visited set. Graph BFS must have one. Without it, a cycle causes the same nodes to be enqueued repeatedly, creating an infinite loop. This is the single most common graph bug.

**When to mark visited:** mark a node as visited when you ADD it to the queue, not when you REMOVE it. If you wait until removal, multiple copies of the same node can sit in the queue simultaneously, wasting time and potentially causing incorrect results.

**Trace through graph: 0-1, 0-2, 1-3, 2-3**

Starting from node 0:

| Step | Queue (front to back) | Process | Add to Queue | Visited |
|---|---|---|---|---|
| Init | [0] | - | - | {0} |

| Step | Queue (front to back) | Process | Add to Queue | Visited |
|---|---|---|---|---|
| 1 | [1, 2] | 0 | 1, 2 | {0, 1, 2} |
| 2 | [2, 3] | 1 | 3 | {0, 1, 2, 3} |
| 3 | [3] | 2 | (3 already visited) | {0, 1, 2, 3} |
| 4 | [] | 3 | (all neighbors visited) | {0, 1, 2, 3} |

BFS order: 0, 1, 2, 3. Nodes at distance 1 from 0 (nodes 1 and 2) are visited before the node at distance 2 (node 3).

**The Shortest Path Guarantee**

In an unweighted graph, BFS visits every node at the minimum number of edges from the source. The first time BFS reaches a node, that path is the shortest. This is why "shortest path in unweighted graph" always means BFS, never DFS.

Time O(V + E), space O(V).

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

## SLIDE 154: DFS on Graphs: Depth-First Exploration

**DFS Template: Recursive**

DFS explores as deep as possible along each branch before backtracking. The recursive implementation is the most natural for interview problems because it follows the recursive mental model from Section 12 directly.

```
void dfs(Map<Integer, List<Integer>> graph, int node, Set<Integer> visi
ted) {
    visited.add(node);

    for (int neighbor : graph.get(node)) {
        if (!visited.contains(neighbor)) {
            dfs(graph, neighbor, visited);
        }
    }
}
```

Three lines of real logic: mark visited, loop over neighbors, recurse on unvisited ones. This simplicity is why recursive DFS is preferred in most interviews.

**DFS Template: Iterative (Stack)**

Replace the recursion with an explicit stack. The structure mirrors BFS exactly, with a stack (LIFO) instead of a queue (FIFO).

```
void dfsIterative(Map<Integer, List<Integer>> graph, int start) {
    Set<Integer> visited = new HashSet<>();
    Deque<Integer> stack = new ArrayDeque<>();
    stack.push(start);

    while (!stack.isEmpty()) {
        int node = stack.pop();
        if (visited.contains(node)) continue;
        visited.add(node);

        for (int neighbor : graph.get(node)) {
            if (!visited.contains(neighbor)) {
                stack.push(neighbor);
            }
        }
    }
}
```

Note: the iterative version marks visited on pop, not on push. This is different from BFS (which marks on enqueue). The reason: a node can be pushed onto the stack multiple times from different neighbors. The visited check on pop handles this correctly.

**BFS vs DFS Decision**

| Criterion | BFS | DFS |
|---|---|---|
| Traversal order | Level by level (closest first) | Deep first (follows one path fully) |
| Data structure | Queue | Stack (explicit or call stack) |
| Shortest path (unweighted) | Yes, guaranteed | No |
| Space (worst case) | O(V) for wide graphs | O(V) for deep graphs |
| Connected components | Either works | Either works |
| Cycle detection | Possible but DFS is standard | Standard approach |
| Topological sort | Not standard | Standard (C5 material) |
| Interview default | Use for shortest path | Use for everything else |

**When Neither Is Clearly Better**

For connectivity problems (is node A reachable from node B? how many components exist?), BFS and DFS perform identically. Choose whichever you can implement more quickly. For most candidates, recursive DFS is fewer lines and less error-prone.

Time O(V + E), space O(V) for both.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

## SLIDE 155: Worked Example: Number of Islands

**Problem: Number of Islands (LeetCode 200)**

Given a 2D grid of '1's (land) and '0's (water), count the number of islands. An island is a group of connected '1's, where connection means horizontally or vertically adjacent (not diagonal).

This is the single most commonly asked graph problem for freshers across all companies. It tests whether you can recognize a grid as a graph and apply DFS or BFS to traverse connected components.

**The Grid-as-Graph Insight**

A grid is an implicit graph. Each cell is a node. Two cells are connected by an edge if they are horizontally or vertically adjacent and both contain '1'. You do not need to build an explicit adjacency list. The adjacency is computed directly from the grid coordinates: each cell (r, c) has up to 4 neighbors at (r-1,c), (r+1,c), (r,c-1), (r,c+1).

**Full Java Solution**

```
public int numIslands(char[][] grid) {
    int count = 0;
    int rows = grid.length;
    int cols = grid[0].length;

    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            if (grid[r][c] == '1') {
                dfs(grid, r, c);
                count++;
            }
        }
    }

    return count;
}

private void dfs(char[][] grid, int r, int c) {
```

```
    if (r < 0 || r >= grid.length || c < 0 || c >= grid[0].length || gr
id[r][c] != '1') {
        return;
    }

    grid[r][c] = '0';

    dfs(grid, r - 1, c);
    dfs(grid, r + 1, c);
    dfs(grid, r, c - 1);
    dfs(grid, r, c + 1);
}
```

The DFS function marks each visited land cell by changing it from '1' to '0'. This uses the grid itself as the visited set, avoiding extra space. Each time the outer loop finds an unvisited '1', it starts a new DFS that marks the entire island, and increments the count.

**Trace through grid:**

```
Initial:          After island 1 DFS:    After island 2 DFS:
1 1 0 0           0 0 0 0                0 0 0 0
1 0 0 0           0 0 0 0                0 0 0 0
0 0 1 1           0 0 1 1                0 0 0 0
0 0 0 1           0 0 0 1                0 0 0 0
```

| Step | Trigger Cell | DFS Marks | Count |
|---|---|---|---|
| Scan finds (0,0) = '1' | (0,0) | (0,0), (0,1), (1,0) | 1 |
| Scan skips water cells, finds (2,2) = '1' | (2,2) | (2,2), (2,3), (3,3) | 2 |

Result: 2 islands. Time O(rows * cols), space O(rows * cols) in the worst case for the recursion stack (a grid of all land).

**Why This Problem Matters**

Number of Islands is the gateway problem for grid-based graph questions. Once you understand this pattern (outer loop finds unvisited nodes, DFS/BFS marks the entire component, count increments), you can solve dozens of variants: max area of island, number of distinct islands, surrounded regions, rotting oranges, shortest path in binary matrix.

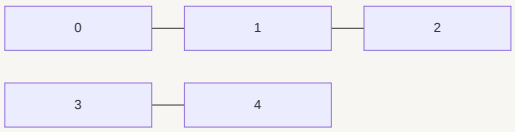
Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

## SLIDE 156: Connected Components

### The General Pattern

Number of Islands is a grid-specific instance of a general graph pattern: counting connected components. The structure is identical for any graph, not just grids.

The approach: iterate over all nodes. For each unvisited node, run DFS or BFS to visit its entire component, then increment the count. The outer loop ensures disconnected components are not missed.




This graph has two connected components: {0, 1, 2} and {3, 4}.

### Full Java Solution

```
public int countComponents(int n, int[][] edges) {
    Map<Integer, List<Integer>> graph = new HashMap<>();
    for (int i = 0; i < n; i++) {
        graph.put(i, new ArrayList<>());
    }
    for (int[] edge : edges) {
        graph.get(edge[0]).add(edge[1]);
        graph.get(edge[1]).add(edge[0]);
    }

    Set<Integer> visited = new HashSet<>();
    int components = 0;

    for (int i = 0; i < n; i++) {
        if (!visited.contains(i)) {
            dfs(graph, i, visited);
            components++;
        }
    }

    return components;
}

private void dfs(Map<Integer, List<Integer>> graph, int node, Set<Integ
```

```
er> visited) {
    visited.add(node);
    for (int neighbor : graph.get(node)) {
        if (!visited.contains(neighbor)) {
            dfs(graph, neighbor, visited);
        }
    }
}
```

**Trace through the graph above (n=5, edges=[[0,1],[1,2],[3,4]]):**

| Outer Loop i | Already Visited? | Action | Components |
|---|---|---|---|
| i = 0 | No | DFS from 0: visits {0, 1, 2} | 1 |
| i = 1 | Yes (visited in i=0's DFS) | Skip | 1 |
| i = 2 | Yes | Skip | 1 |
| i = 3 | No | DFS from 3: visits {3, 4} | 2 |
| i = 4 | Yes (visited in i=3's DFS) | Skip | 2 |

Result: 2 components. Time O(V + E), space O(V).

**The Outer Loop Is Essential**

A common mistake is running BFS or DFS from only one starting node and assuming it visits the entire graph. If the graph is disconnected, a single traversal only reaches one component. The outer loop over all nodes catches every component. This applies to Number of Islands (outer loop over all grid cells), connected components (outer loop over all nodes), and cycle detection (next slide).

**Variant Problems**

| Problem | Same Structure, Different Detail |
|---|---|
| Number of Islands (LeetCode 200) | Grid instead of explicit graph, DFS marks cells |
| Number of Provinces (LeetCode 547) | Adjacency matrix input, same component counting |
| Max Area of Island (LeetCode 695) | Track component size during DFS, return maximum |
| Accounts Merge (LeetCode 721) | Nodes are emails, edges from shared accounts |

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

## SLIDE 157: Cycle Detection in Graphs

### Why Cycle Detection Matters

Cycles indicate circular dependencies, infinite loops, or invalid structures. "Can all courses be completed?" is a cycle detection problem on a directed graph of prerequisites. "Is this graph a valid tree?" is equivalent to asking: is it connected with no cycles?

### Undirected Graph: DFS with Parent Tracking

During DFS on an undirected graph, every edge is traversed in both directions (A to B and B to A). This means when processing node B, its neighbor A is already visited, but that does not indicate a cycle because we just came from A. The fix: track the parent of each node. If a neighbor is visited AND is not the parent, a cycle exists.

```
public boolean hasCycle(Map<Integer, List<Integer>> graph, int n) {
    Set<Integer> visited = new HashSet<>();

    for (int i = 0; i < n; i++) {
        if (!visited.contains(i)) {
            if (dfsCycle(graph, i, -1, visited)) return true;
        }
    }

    return false;
}

private boolean dfsCycle(Map<Integer, List<Integer>> graph, int node, i
nt parent,
                         Set<Integer> visited) {
    visited.add(node);

    for (int neighbor : graph.get(node)) {
        if (!visited.contains(neighbor)) {
            if (dfsCycle(graph, neighbor, node, visited)) return true;
        } else if (neighbor != parent) {
            return true;
        }
    }

    return false;
}
```

The parent parameter is the node we came from. When a neighbor is visited and is not the parent, we have found a back edge, which proves a cycle exists.

**Trace through graph with edges: [0-1, 1-2, 1-3, 2-3]**

```
0 --- 1 --- 2
      |   /
      3 -
```

| Step | Node | Parent | Neighbor | Status | Action |
|---|---|---|---|---|---|
| 1 | 0 | -1 | 1 | Unvisited | Recurse into 1 |
| 2 | 1 | 0 | 0 | Visited, IS parent | Skip (not a cycle) |
| 3 | 1 | 0 | 2 | Unvisited | Recurse into 2 |
| 4 | 2 | 1 | 1 | Visited, IS parent | Skip |
| 5 | 2 | 1 | 3 | Unvisited | Recurse into 3 |
| 6 | 3 | 2 | 1 | Visited, NOT parent | Cycle detected |

At step 6, node 3 sees neighbor 1 which is visited but is not 3's parent (parent is 2). This is a back edge proving the cycle 1-2-3-1 exists.

**Directed Graph: Three-State Coloring (Preview)**

Undirected cycle detection uses a parent check. Directed graphs require a different approach because edges are one-way. The standard method uses three states: unvisited (white), in the current DFS path (gray), and fully processed (black). If DFS reaches a gray node, a back edge exists and a cycle is confirmed. This technique is the foundation for topological sort, covered in C5.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

## SLIDE 158: Wrong vs Right: Graph Mistakes

**Mistake 1: Forgetting the visited set**

Wrong: running BFS or DFS on a graph without tracking visited nodes. On a graph with cycles, this causes infinite traversal. The program loops forever or runs out of memory.

Better: always declare a visited set before any graph traversal. For grid problems, you can use the grid itself as the visited tracker by modifying cell values (as in Number of Islands), but you must still have a visited mechanism.

**Mistake 2: Running BFS/DFS from only one node on a disconnected graph**

Wrong: calling bfs(graph, 0) and assuming all nodes are reachable. If the graph has multiple components, nodes in other components are never visited.

Better: wrap the traversal in an outer loop over all nodes. Check if each node has been visited before starting a new BFS/DFS. This is the connected component pattern from Slide 156.

**Mistake 3: Using adjacency matrix for sparse graphs**

Wrong: creating a V x V matrix for a graph with V = 10,000 nodes and only 20,000 edges. The matrix wastes O(V^2) = 100 million cells of memory when the adjacency list would use O(V + E) = 30,000 entries.

Better: default to adjacency list. Use matrix only when V is small (under a few hundred) or when you need O(1) edge lookup for a specific algorithm.

**Mistake 4: Confusing directed and undirected edge addition**

Wrong: adding edges in only one direction for an undirected graph, or adding both directions for a directed graph. The first causes nodes to be unreachable. The second creates false edges that do not exist.

Better: always ask "directed or undirected?" before building the graph. Undirected: two add operations per edge. Directed: one add operation per edge.

**Mistake 5: Marking visited on dequeue instead of enqueue (BFS)**

Wrong: checking visited when removing a node from the queue. Between the time a node is enqueued and dequeued, other paths may enqueue it again. The node gets processed multiple times.

Better: mark visited immediately when adding to the queue. This ensures each node is enqueued at most once.

**Mistake 6: Not clarifying adjacency for grid problems**

Wrong: assuming diagonal adjacency when the problem means 4-directional, or vice versa.

Better: ask the interviewer: "When you say adjacent, do you mean 4-directional (up, down, left, right) or 8-directional (including diagonals)?" Default is 4-directional unless stated otherwise.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

## SLIDE 159: How to Present Graph Solutions in an Interview

**Step 1: Clarify the Graph**

Before writing any code, ask three questions:

"Are the edges directed or undirected?" This determines how you build the adjacency list and which cycle detection method to use.

"Are the edges weighted?" This determines whether BFS gives shortest paths or whether you need Dijkstra.

"Can the graph be disconnected?" This determines whether you need the outer loop over all nodes.

These three questions take 15 seconds and demonstrate graph literacy. Most freshers skip them and run into problems midway through implementation.

**Step 2: State the Representation**

"I'll represent this as an adjacency list since the graph is sparse. Each node maps to a list of its neighbors."

For grid problems: "I'll treat the grid as an implicit graph where each cell is a node and adjacent land cells are connected by edges. I don't need to build an explicit adjacency list."

**Step 3: State the Traversal**

"I need shortest path, so I'll use BFS." Or: "I need to find all connected components, so I'll use DFS with an outer loop."

If the interviewer asks why BFS over DFS (or vice versa), give the concrete reason: "BFS guarantees shortest path in unweighted graphs because it explores level by level."

**Step 4: Name the Complexity**

Graph algorithms are always O(V + E) for both BFS and DFS. State this explicitly: "I visit every node once and traverse every edge once, so the time complexity is O(V + E). The space is O(V) for the visited set and the queue (or stack)."

For grid problems, translate: "V is rows times cols, and E is at most 4 times V (each cell has at most 4 neighbors), so the time is O(rows times cols)."

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

## SLIDE 160: Interview Practice: Graphs

**Practice Prompt**

"Given a 2D grid of '1's (land) and '0's (water), count the number of islands. An island is surrounded by water and is formed by connecting adjacent lands horizontally or vertically."

**Expected Walkthrough**

Step 1, Understand: a 2D char grid, '1' is land, '0' is water. Adjacent means 4-directional (not diagonal). An island is a connected group of '1's. Count the number of such groups.

Step 2, Pattern: this is connected component counting on an implicit graph. Each '1' cell is a node. Adjacent '1' cells are edges.

Step 3, Approach: iterate over every cell. When an unvisited '1' is found, run DFS to mark the entire island as visited (change '1' to '0'), then increment the count. The outer loop handles multiple islands.

Step 4, Implement: the solution from Slide 155.

Step 5, Verify: trace through a small grid with 2-3 islands. Confirm each DFS marks exactly one island. State O(rows * cols) time and space.

**Follow-Up: "What if you can't modify the input grid?"**

Use a separate boolean[][] visited array instead of modifying the grid in place. Same logic, same complexity, but O(rows * cols) explicit extra space for the visited array.

**Follow-Up: "What if you need to find the largest island?"**

Track the DFS return value: count cells visited in each DFS call, return the count, and track the maximum across all calls. Same traversal structure with one additional piece of bookkeeping.

**Common Fresher Mistakes**

| Mistake | What Goes Wrong | Fix |
|---|---|---|
| Not marking visited during DFS | Same cell processed multiple times, count inflated | Mark cell before recursing (grid[r][c] = '0' or visited[r][c] = true) |
| Assuming diagonal adjacency | Counts fewer or more islands than expected | Clarify 4-directional vs 8-directional with the interviewer |
| No bounds checking in DFS | ArrayIndexOutOfBoundsException on grid edges | First line of DFS: check r and c are within bounds |
| Single DFS without outer loop | Only finds the first island, misses disconnected ones | Outer loop over all cells triggers DFS for each new island |

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

## SLIDE 161: Section Cheat Sheet

**Representation Decision**

| Factor | Adjacency List | Adjacency Matrix |
|---|---|---|
| Space | O(V + E) | O(V^2) |
| Edge lookup | O(degree) | O(1) |
| Interview default | Yes | Only for small/dense graphs |
| Grid problems | Implicit (no explicit build needed) | Not applicable |

### BFS Template

```
Queue + visited set
Mark visited on ENQUEUE (not dequeue)
Process neighbors, enqueue unvisited ones
Guarantees shortest path in unweighted graphs
Time O(V + E), space O(V)
```

### DFS Template

```
Recursive: mark visited, recurse on unvisited neighbors
Iterative: stack, mark visited on POP, push unvisited neighbors
Does NOT guarantee shortest path
Time O(V + E), space O(V)
```

### BFS vs DFS Decision

| Problem Type | Use | Why |
|---|---|---|
| Shortest path (unweighted) | BFS | Level-by-level guarantees minimum distance |
| Connected components | Either | Both visit all reachable nodes |
| Cycle detection (undirected) | DFS | Parent tracking is natural with DFS |
| Cycle detection (directed) | DFS | Three-state coloring requires DFS path tracking |
| Topological sort | DFS | Post-order gives reverse topological order (C5) |
| Grid flood fill | DFS (recursive) | Cleaner code, natural recursion |

### Pattern Recognition Signals

| Signal in Problem Statement | Maps To | Template |
|---|---|---|
| "count islands", "connected components" | Outer loop + DFS/BFS | Component counting |
| "shortest path", "minimum steps" (unweighted) | BFS with distance tracking | BFS level by level |
| "can all courses be completed", "valid ordering" | Cycle detection on directed graph | DFS coloring (C5) |

| Signal in Problem Statement | Maps To | Template |
|---|---|---|
| "is this a valid tree" | Connected + no cycles | Component count = 1 and E = V - 1 |
| "flood fill", "surrounded regions" | Grid DFS/BFS | Mark connected cells |
| Grid given as input | Implicit graph | No adjacency list build needed |

### Three Questions Before Any Graph Problem

1. Directed or undirected?
2. Weighted or unweighted?
3. Can it be disconnected?

### How This Section Connects to Other Sections

| Section | Connection |
|---|---|
| Stacks and Queues (Section 10) | BFS uses queue, iterative DFS uses stack. The data structure choice is the only difference between the two traversals. |
| Recursion (Section 12) | Recursive DFS is a direct application of the recursive template: base case (already visited or out of bounds), recursive case (visit neighbors) |
| Trees (Section 15) | Tree traversals are graph traversals without a visited set. BFS on a tree is level-order traversal. DFS on a tree is preorder/inorder/postorder. |
| DP (Section 17) | Some grid DP problems look like graph problems but use memoized recursion instead of BFS/DFS. Knowing which tool to reach for comes from recognizing whether the problem has overlapping subproblems (DP) or just connectivity (graph traversal). |

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina